TLP:CLEAR | UNRESTRICTED DISTRIBUTION

THREAT INTELLIGENCE REPORT

FamousSparrow / SparrowDoor

Dynamic and Static Analysis of a PRC-Attributed Loader Component

Report ID	CTI-003 | SparrowDoor
Report Date	March 16, 2026
Analyst	SerapHim
Analyst Role	Independent Cyber Threat Intelligence
Sample SHA256	`8dfaa1f579de14bca8bb27c54a57dd87646a835969766ca9ddb81ecd9329f4e4`
Sample Filename	K7UI.dll (masquerading)
Threat Actor	FamousSparrow / Salt Typhoon / Earth Estries
Attribution	PRC - HIGH confidence (external reporting)
Malware Family	SparrowDoor - Legacy Variant (2019-2022)
Sample Source	ESET malware-ioc (github.com/eset/malware-ioc/famoussparrow)
Analysis Environment	isolated-CTI private switch, network-gapped
Analysis Tools	file, strings, objdump, Python3 Shannon entropy, Ghidra 10.4
TLP Classification	TLP:CLEAR - Unrestricted Distribution
MITRE ATT&CK	T1574.002, T1027, T1055.001, T1562.001, T1140, T1059.003, T1071.001

1. Executive Summary

This report documents independent static analysis of a FamousSparrow loader component (SHA256: 8dfaa1f579..., filename K7UI.dll), conducted in a network-isolated Parrot OS environment using file, strings, objdump, Python3 Shannon entropy calculation, and Ghidra 10.4. The sample was obtained from the ESET malware-ioc public repository. Technical findings are combined with external vendor intelligence to document the sample's architecture, obfuscation mechanisms, and anti-analysis techniques.

Twenty technical findings (F1-F20) are documented from direct sample analysis. Four findings are not present in prior public vendor reporting at time of publication: an inverted anti-sandbox technique that terminates the process when no debugger is detected (F20), a three-table character substitution

system for case-insensitive command processing without Windows API calls (F12-F15), anomalous .text section entropy indicating code obfuscation (F11), and an indirect call density of 113 FF 15 sites across a 26KB binary (F16). These are labeled [PHIM] throughout the report.

The sample corresponds to the legacy SparrowDoor variant (2019-2022) based on its single-stage loader architecture, basic reflective loading mechanism, and absence of parallelization capability. It functions as a Stage 1 component whose sole purpose is to load MpSvc.dll from disk into memory via reflective injection and transfer execution. No network capability, registry operations, or embedded C2 configuration are present in Stage 1.

2. Methodology

2.1 Scope and Objectives

This report set out to accomplish two objectives: (1) conduct independent static and behavioral analysis of a publicly available FamousSparrow sample to identify technical characteristics not documented in prior vendor reporting; and (2) correlate those findings with external vendor intelligence to construct an accurate assessment of the sample's architecture, evasion design, and operational role.

2.2 Secondary Analysis -- External Intelligence

External vendor intelligence sourced from ESET (2021, 2025), UK NCSC (2022), Trend Micro (2024), Trustwave SpiderLabs (2024), Microsoft, FBI, and CISA (2024). All claims are explicitly attributed and were not independently replicated by SerapHim.

2.3 Confidence Level Scale

Level	Definition
HIGH	Supported by direct technical evidence or corroborated open source reporting
MODERATE	Supported by circumstantial evidence or pattern correlation
LOW	Inferred from limited indicators - requires additional evidence to confirm

3. Technical Analysis

Findings F1-F20 represent original SerapHim analysis conducted directly against the sample. Findings marked [PHIM] were not documented in prior public vendor reporting at time of publication.

3.1 Sample Identification

F1 - PE Header

Attribute	Value
Type	PE32 executable, 32-bit DLL, Intel i386
Subsystem	Windows 5.00 (Windows 2000/XP era compatibility)
Sections	5 (.text, .rdata, .data, .rsrc, .reloc)

The binary is a 32-bit DLL compiled for Intel i386. The Windows 5.00 subsystem header is deliberate: it ensures the binary runs via WOW64 on all x64 Windows systems while reducing the likelihood of matching version-specific detection signatures built against modern PE headers. The 32-bit architecture provides maximum compatibility across heterogeneous victim environments, including hotel management systems, POS infrastructure, and legacy government deployments.

F2 - DLL Sideloading via K7UI.dll Masquerade

Attribute	Value
Export name	K7UI.dll
Exported functions	K7UI_1, K7UI_13

The export table is fully falsified to claim the identity of a K7 Antivirus component. This is not merely a filename match - the entire export table asserts K7 identity. When a system calls the legitimate K7UI, SparrowDoor answers. K7 Computing is a Chennai-based security vendor with primary market penetration in India, Southeast Asia, and parts of the Middle East. Security software operates with elevated trust and is unlikely to trigger suspicion when observed loading in an enterprise environment.

F3 - MpSvc.dll Reference (Secondary Masquerade)

The string MpSvc.dll is present in the binary. MpSvc.dll is a Windows Defender Malware Protection service component. This represents a second layer of identity masquerade: K7 Antivirus at the export table level, Windows Defender at the string reference level. The C2 configuration and Stage 2 payload are loaded from MpSvc.dll at runtime. The dual-masquerade architecture means any casual inspection encounters a plausible legitimate identity before reaching the malicious logic.

3.2 Obfuscation Mechanisms

F4 - Character Substitution Tables (Static View)

Three lookup table variants are embedded in the binary:

```
Table 1: ...ABCDEFGHIJKLMNOPQRSTUVWXYZ...ABCDEFGHIJKLMNOPQRSTUVWXYZ...
Table 2: ...ABCDEFGHIJKLMNOPQRSTUVWXYZ...abcdefghijklmnopqrstuvwxyz...
Table 3: ...ABCDEFGHIJKLMNOPQRSTUVWXYZ...abcdefghijklmnopqrstuvwxyz...
```

Strings with operational significance C2 addresses, registry paths, commands are not visible as plaintext. They are decoded at runtime using these tables. The substitution operates as a ROT/Caesar-style cipher with three variants covering different case transformation modes. Full address-level analysis is in F12-F15.

F12 - Substitution Table Memory Addresses [PHIM]

Method: Ghidra Memory Search string !"#$%&, encoding US-ASCII, Search All.

```
10008898 -- Table variant 1 (Identity)
10008ea1 -- Table variant 2 (Uppercase-to-Lowercase)
10009021 -- Table variant 3 (Lowercase-to-Uppercase)
```

All three tables reside in .rdata (0x10008000-0x10009xxx). Byte-level confirmation at Table 1 base address:

```
10008898: 21h = '!'
10008899: 22h = '"'
1000889a: 23h = '#'
1000889b: 24h = '$'
1000889c: 25h = '%'
1000889d: 26h = '&'
```

F13 - Table 1 (0x10008898): Identity Table [PHIM]

Range: 0x21 (!) through 0x7E (~). Each byte maps to itself, no substitution occurs. This table functions as a passthrough baseline, used for validating that input is printable ASCII before processing by Tables 2 or 3. It is not an encoder.

F14 - Table 2 (0x10008ea1): Uppercase-to-Lowercase Converter [PHIM]

Deviation begins at: 10008ec1 (offset 0x40 from table start)

```
0x41 'A' -> 0x61 'a'
0x42 'B' -> 0x62 'b' ... 0x5A 'Z' -> 0x7A 'z'
Non-alpha range: identity (no change)
```

A pure case normalization table converting A-Z to a-z. Allows SparrowDoor to normalize C2 commands to lowercase before processing, enabling case-insensitive matching regardless of how the operator sends them. Critically, this is achieved without calling Windows API functions such as CompareStringA or lstrcmpiA - both would appear in the import table and are commonly flagged by security tooling. This is deliberate import-evasion engineering.

F15 - Table 3 (0x10009021): Lowercase-to-Uppercase Converter [PHIM]

Deviation begins at: 10009061 (offset 0x40 from table start)

```
0x61 'a' -> 0x41 'A'
0x62 'b' -> 0x42 'B' ... 0x7A 'z' -> 0x5A 'Z'
Non-alpha range: identity (no change)
```

The exact inverse of Table 2. Together, the three tables form a triplet normalization system: Table 1 for passthrough/validation, Table 2 for normalizing to lowercase, Table 3 for normalizing to uppercase. Bidirectional case-insensitive matching without any Windows string API call. No equivalent triplet system has been documented in ESET, UK NCSC, Trend Micro, or Microsoft FamousSparrow reporting.

3.3 Anti-Analysis Techniques

F5 - IsDebuggerPresent Import

IsDebuggerPresent (KERNEL32.dll) is present in the import table. SparrowDoor actively detects whether it is executing inside a debugging environment. The full implementation and critical inversion of conventional anti-debug logic are documented in F20.

F20 - DllMain Entry Point: Inverted Anti-Sandbox Logic [PHIM]

Method: Ghidra disassembly and decompilation of entry function at 100016a1.

Entry point flow (100016a1):

```
100016a6  CMP dword ptr [EBP + param_2], 0x1  ; Check fdwReason == DLL_PROCESS_ATTACH
100016aa  JNZ LAB_100016b1                    ; Skip if not PROCESS_ATTACH
100016ac  CALL FUN_10003c1d                    ; Main initialization
100016ba  CALL FUN_100015ab                    ; Secondary initialization
```

Anti-sandbox logic (FUN_100016c4):

Phase 1 -- Environment detection:

```
1000177f  CALL dword ptr [IsDebuggerPresent]  ; 0 = no debugger, 1 = present
10001785  MOV [DAT_1000ace8], EAX             ; Store result
```

Phase 3 -- Anti-sandbox termination logic:

```
100017a5  CMP dword ptr [DAT_1000ace8], 0x0  ; Check IsDebuggerPresent
100017ac  JNZ LAB_100017b6                    ; Debugger present: skip termination
100017b6  PUSH 0xc0000409                     ; STATUS_STACK_BUFFER_OVERRUN
100017bb  CALL GetCurrentProcess
100017c2  CALL TerminateProcess                ; KILL THE PROCESS
```

CRITICAL FINDING - Inverted Logic:

- Debugger detected (IsDebuggerPresent == 1): jump to LAB_100017b6, skip termination, malware runs normally
- No debugger detected (IsDebuggerPresent == 0): fall through to TerminateProcess, malware terminates immediately

This is not traditional anti-debug behavior, which terminates when a debugger is present. This is anti-sandbox behavior. Automated sandboxes (Cuckoo, ANY.RUN, Joe Sandbox) do not attach debuggers. IsDebuggerPresent returns 0 in those environments. SparrowDoor detects the absence of a debugger, the standard sandbox condition, and terminates. The sandbox logs a crash on startup

and classifies the sample as non-functional. A human analyst working with a debugger attached proceeds normally.

The exception code 0xc0000409 (STATUS_STACK_BUFFER_OVERRUN) masquerades the deliberate termination as a triggered compiler security feature, not a malware self-kill. Two 32-bit cookie values are loaded into stack locals before the check: 0xBB40E64E and 0x44BF19B1, candidates for stack canaries or encryption keys.

Three hypotheses remain unresolved without dynamic analysis. Hypothesis A: FUN_100016c4 is operational anti-sandbox logic executing in production under specific initialization conditions. Hypothesis B: deliberate dead code designed to waste analyst time. Hypothesis C: conditional compilation artifact present in test builds to prevent sandbox submission, absent in production. Hypothesis A is most probable given the deliberate exception code masquerade.

CONFIDENCE: MODERATE - technique confirmed from disassembly. Operational function unresolved without dynamic analysis.

3.4 Loader Architecture

F6 - Minimal Import Profile (KERNEL32.dll Only)

Attribute	Value
Import count	1 DLL (KERNEL32.dll), approximately 60 functions
Notable absent	Winsock, WinHTTP, WinINet, Advapi32, Ws2_32

The absence of network, registry, and cryptographic library imports from the IAT is not an oversight. All advanced OS calls are loaded dynamically via LoadLibraryA + GetProcAddress at runtime. Static analysis tools and AV scanners that rely on import table inspection observe only the KERNEL32 footprint and draw no conclusions about network or registry capability.

F7 -- Reflective Loading Pattern

```
VirtualAlloc
VirtualProtect
ReadFile
GetProcAddress
LoadLibraryA
```

This combination is the classical signature for reflective DLL loading. MpSvc.dll is loaded from disk via CreateFileA + ReadFile, allocated to memory via VirtualAlloc, marked executable via VirtualProtect, and its exports resolved via manual PE parsing + GetProcAddress. Stage 2 operates entirely in memory and does not appear on disk as a standalone file during execution, defeating file-based AV scanning.

F16 - Indirect Call Density: 113 FF 15 Sites [PHIM]

Attribute	Value

Method	Ghidra Memory Search -- Hex FF 15, Search All
Result	113 indirect call locations in .text section
Address range	10001042 - 100070df
Density	~1 indirect call per 230 bytes in a 26KB binary

FF 15 is the x86 opcode for CALL [mem] - an indirect call through a pointer. 113 occurrences across a 26KB binary represents very high density. Static analysis tools that inspect only the IAT observe approximately 60 visible KERNEL32 functions. The 113 indirect call sites represent the hidden majority of actual OS capability -- network operations, registry access, cryptographic functions, all resolved and called at runtime without static visibility. This architecture makes signature-based import scanning systematically incomplete for this sample.

F17 - LoadLibraryA Reference: Single DATA Pointer

Result: 1 reference only - DATA reference type, located in .data section. LoadLibraryA is not called directly from any code execution path. Its address is stored as a pointer and is called indirectly through one of the 113 FF 15 sites identified in F16. Consistent with a centralized function pointer dispatcher architecture.

F18 -- Complete IAT Structure: 60 KERNEL32 Functions

Method: Ghidra Listing navigation to 10008000 (IAT start). Section .rdata, range 10008000-10009bff.

```
Dynamic Loading:  100080d0 LoadLibraryA      10008070 GetProcAddress
Memory Mgmt:      10008010 VirtualAlloc      1000801c VirtualProtect    10008054
VirtualFree
File Operations:  10008004 CreateFileA       1000800c ReadFile          10008074 WriteFile
Anti-Analysis:    10008048 IsDebuggerPresent
TLS (threading):  10008080 TlsAlloc          1000807c TlsGetValue       10008084
TlsSetValue
Synchronization:  10008060 EnterCriticalSection  1000805c LeaveCriticalSection
```

The IAT is deliberately minimal yet complete for Stage 1 loader functionality. Network, registry, and cryptographic functions are entirely absent these reside in Stage 2. The presence of TLS and critical section primitives confirms multi-threaded architecture. String operation functions directly support the substitution table system documented in F12-F15.

F19 - Dynamic Loading String Search: Negative Result

Searched: ws2_32, winhttp, wininet, ntdll, advapi32

Result: NONE FOUND

K7UI.dll is a pure Stage 1 loader with tightly bounded scope. No DLL names for dynamic loading, no function name strings for GetProcAddress resolution, no network library references. The separation of concerns is architectural and deliberate: Stage 1 loads Stage 2 from disk and transfers execution. Stage 2 carries all operational capability.

F8 - No Visible C2 Configuration

No IP address, domain, or URL is present as a plaintext string in the binary. C2 configuration is entirely in Stage 2 (MpSvc.dll), loaded into memory at runtime and decrypted using the substitution table system (F4, F12-F15). Static analysis of Stage 1 alone is structurally incapable of recovering C2 information.

3.5 Entropy Analysis

F10 - Overall Entropy: 6.14 / 8.0

Method: Python3 Shannon entropy calculation. Result: 6.1401536847751552 / 8.0. Scale 0-8, where 8.0 is fully random (encrypted). 6.14 indicates mixed content partially obfuscated sections coexist with sparse data sections. Consistent with a loader architecture where the code section is obfuscated (F11) but payload data is not embedded.

F11 -- Per-Section Entropy [PHIM]

Section	Entropy	Assessment
.text	6.5937	ANOMALOUS - elevated for a code section (typical range: 5.0-5.5)
.rdata	5.1229	Normal
.data	2.2542	Very low - mostly zeros / padding
.rsrc	4.8788	Normal
.reloc	4.5937	Normal

The .text entropy of 6.59 is a significant anomaly indicating code obfuscation or self-modifying logic within the execution path. The .data entropy of 2.25 confirms the Stage 2 payload is not embedded - MpSvc.dll is loaded from disk separately. These entropy profiles have not been documented in prior FamousSparrow vendor reporting.

3.6 Section Structure

F9 - Section Analysis

Section	Size	Offset	Flags
.text	26KB (0x6624)	0x400	CODE, READONLY
.rdata	7KB (0x1bd0)	0x6c00	DATA, READONLY
.data	3.5KB (0x0e00)	0x8800	DATA
.rsrc	436b (0x01b4)	0x9600	DATA, READONLY
.reloc	3KB (0x0c52)	0x9800	DATA, READONLY

The .rsrc section is near-empty at 436 bytes, containing no icon, version info, or meaningful resources. Deliberate: resource sections are a primary fingerprinting target for static analysis tools. Minimal .rsrc reduces the static signature surface. The .reloc section with approximately 150 fixups is

high relative to binary size, consistent with position-independent code designed to execute at arbitrary memory addresses, supporting the reflective loading mechanism in F7.

4. Architectural Summary

Based on F1-F20, the sample is a Stage 1 Loader Component. Its architecture is built around one objective: load Stage 2 from disk, inject it into memory, and transfer execution, while remaining as invisible as possible to static analysis and automated sandbox environments.

Component	Role	Key Findings
Stage 1 (K7UI.dll)	Loader: masquerade, anti-sandbox, reflective inject MpSvc.dll	F1-F9, F11-F20
Stage 2 (MpSvc.dll)	Backdoor: C2, command execution, credential harvesting	External reporting only, not analyzed

The separation of concerns is architectural and deliberate. Stage 1 presents a minimal import profile only KERNEL32.dll, no network or registry functions visible. The 113 indirect call sites mask the actual runtime call pattern. The .text section obfuscation prevents signature matching against the execution logic. The inverted anti-sandbox check defeats automated analysis. Stage 2 never appears on disk as a standalone file during execution.

Stage 2 capabilities from external reporting (UK NCSC, ESET): remote command execution, file upload/download, screenshot capture, keylogging, process management, file system change monitoring, C2 via HTTPS port 443, XOR encryption with key hH7@83#mi, configurable beaconing intervals.

5. Indicators of Compromise

5.1 File Indicators

Type	Indicator	Notes
SHA256	8dfaa1f579de14bca8bb27c54a57dd87646a835969766ca9ddb81ecd9329f4e4	K7UI.dll - Stage 1 loader
File path	K7UI.dll	DLL sideloading target - K7 Antivirus masquerade
Export functions	K7UI_1, K7UI_13	Falsified export table
String	MpSvc.dll	Stage 2 component reference

Exception code	0xc0000409	Anti-sandbox masquerade (STATUS_STACK_BUFFER_OVERRUN)
Cookie values	0xBB40E64E, 0x44BF19B1	Stack canary / encryption key candidates

5.2 Network Indicators (External Reporting)

Type	Indicator	Notes
IP	194.165.16[.]69	SparrowDoor C2 (ESET)
IP	198.12.75[.]112	SparrowDoor C2 (ESET)
Domain	cdn.kkxx888666[.]com	DRBControl overlap - Metasploit C2 on FamousSparrow victim
Protocol	HTTPS port 443	C2 communication (UK NCSC)
XOR key	hH7@83#mi	Stage 2 C2 encryption key (UK NCSC)

5.3 Behavioral Indicators

Type	Indicator	Notes
Anti-sandbox [PHIM]	Terminates if IsDebuggerPresent == 0	Inverted logic - F20
Substitution tables [PHIM]	.rdata pattern at 0x10008898, 0x10008ea1, 0x10009021	YARA candidate - F12-F15
Indirect calls [PHIM]	113 FF 15 sites in .text, range 10001042-100070df	Anti-detection architecture - F16

6. MITRE ATT&CK Mapping

ID	Technique	Observed
T1574.002	DLL Side-Loading	K7UI.dll masquerading as K7 Antivirus component
T1027	Obfuscated Files or Information	Substitution table obfuscation; .text entropy 6.59
T1055.001	Process Injection: DLL Injection	Reflective loading of MpSvc.dll into memory
T1562.001	Impair Defenses: Disable or Modify Tools	Inverted anti-sandbox logic via IsDebuggerPresent
T1140	Deobfuscate / Decode Files or Information	Runtime decryption via substitution tables
T1059.003	Command Interpreter: Windows Command Shell	Stage 2 remote command execution (external reporting)

T1071.001	Application Layer Protocol: Web Protocols	HTTPS C2 on port 443 (UK NCSC)
T1195.001	Supply Chain Compromise: Software Dependencies	ProxyLogon exploitation targeting Exchange (2021)

7. External Intelligence & Attribution

Attribution disclaimer: All attribution claims in this section are based on external vendor reporting. SerapHim analysis (F1-F20) makes no independent attribution, scope is limited to the technical sample.

7.1 Threat Actor Aliases

Alias	Source	Year
FamousSparrow	ESET	2021
Salt Typhoon	Microsoft	2024
UNC2286	Mandiant	Undisclosed
GhostEmperor	Kaspersky	2020-2021
Earth Estries	Trend Micro	2024

PRC affiliation is confirmed by FBI and CISA joint statements (2024). ESET (2025) and Microsoft (2024) disagree on whether FamousSparrow and GhostEmperor are the same actor or distinct groups with observed overlaps. This report treats them as the same actor consistent with the Microsoft assessment.

7.2 Campaign Timeline

Period	Activity
August 2019	First documented SparrowDoor deployment - hotels worldwide
March 3, 2021	ProxyLogon exploitation begins - one day after patch release (March 2, 2021)
2019-2022	Legacy SparrowDoor variant active - hotel infrastructure focus
2023-present	Pivot to telecommunications and ISP targeting; modular variant deployed
November 2024	Trend Micro / Trustwave SpiderLabs disclosure
March 2025	ESET updated analysis - two new modular variants documented
2024-2025	FBI / CISA joint advisory; WSJ reporting on US telecom breaches

7.3 SparrowDoor Variant Comparison

Attribute	Legacy (2019-2022)	Modular (2024-present)
Architecture	Single-stage loader	Multi-stage, plugin system
Command processing	Sequential	Parallel (concurrent execution)
Code quality	Basic	Improved - marked progress per ESET
Sample analyzed	Yes -- this report	No - not analyzed

The sample analyzed (8dfaa1f579...) corresponds to the legacy variant based on its single-stage loader architecture (F19), basic reflective loading mechanism (F7, F18), and absence of parallelization capability. The modular 2024 variant represents significant evolution.

7.4 SerapHim Findings vs External Reporting

Finding	SerapHim (F1-F20)	External Reporting	Status
DLL Sideloading	K7UI.dll masquerade confirmed (F2)	ESET: K7 AntiVirus abuse	Full match
MpSvc.dll Stage 2	String reference (F3)	UK NCSC: libcurl.dll + libhost.dll	Variant difference
Inverted anti-sandbox	Inverted IsDebuggerPresent (F20)	Not documented	[PHIM]
Substitution tables	Triplet normalization system (F12-F15)	Not documented	[PHIM]
Network capability	Absent from Stage 1 (F19)	UK NCSC: C2 in Stage 2	Consistent
.text entropy anomaly	6.59/8.0 (F11)	Not documented	[PHIM]
Indirect call density	113 FF 15 sites (F16)	Not documented	[PHIM]

8. Pivot Analysis

All pivot work conducted using passive open-source intelligence only. No active probing of infrastructure was performed.

8.1 Sample Hash

The SHA256 8dfaa1f579de14bca8bb27c54a57dd87646a835969766ca9ddb81ecd9329f4e4 is sourced from the ESET malware-ioc GitHub repository. No occurrence was found in prior CTI reporting outside the ESET disclosure at time of analysis. VirusTotal historical data, first detection

date, and vendor classification spread were not accessible at the analyst's access level - deferred to future analysis with commercial platform access. The hash is now publicly documented in CTI-003 as a legacy variant (2019-2022) identifier.

8.2 C2 Infrastructure

Two C2 server IPs are documented in ESET malware-ioc: 194.165.16[.]69 and 198.12.75[.]112. ASN, hosting context, historical passive DNS, certificate metadata, and port scan history were not accessible at the analyst's access level (requires PassiveTotal, DomainTools, Shodan, or Censys paid access). The domain cdn.kkxx888666[.]com was observed as a Metasploit C2 on a FamousSparrow-compromised machine, attributed to the DRBControl group, consistent with tool-sharing and contractor overlap patterns in external reporting.

8.3 K7UI Export Pattern as Tracking Artifact

The export naming pattern K7UI_[digit] is a YARA-suitable fingerprint. K7 Computing's market penetration is concentrated in India (primary), Southeast Asia, and a secondary MENA subset. It is not widely deployed in Europe, the Americas, or Africa. The UK NCSC sample uses libcurl.dll and libhost.dll naming, generic FOSS names with no regional identity. This pattern is consistent with FamousSparrow deploying region-adapted masquerade variants rather than a universal naming scheme.

CONFIDENCE: MODERATE - K7 market penetration confirmed. Variant-specific deployment is inferred from naming and victim geography correlation.

8.4 Substitution Table Byte Pattern (YARA Candidate)

The three-table sequence beginning at 0x10008898 is a candidate for a file-based YARA detection rule targeting the triplet pattern across FamousSparrow variants. Not documented in any prior public reporting, would be a high-specificity signature if the same pattern appears in the 2024 modular variant. Cross-sample search requires access to a large malware database, deferred to future analysis.

8.5 ProxyLogon +1 Day Exploitation Timeline

Date	Event
March 2, 2021	Microsoft releases ProxyLogon patches (CVE-2021-26855, -26857, -26858, -27065)
March 3, 2021	FamousSparrow begins ProxyLogon exploitation - one day post-release

Hafnium had zero-day access before public disclosure. FamousSparrow was among the fastest post-disclosure actors, beginning exploitation within 24 hours of patch release. This suggests either advance knowledge of vulnerability details before public release, or a rapid exploit development

capability of under 24 hours. If this pattern holds across multiple disclosures, it becomes a behavioral signature for FamousSparrow.

CONFIDENCE: HIGH (timeline confirmed by ESET telemetry). MODERATE (advance knowledge hypothesis - circumstantial).

8.6 Pivot Summary and Tracking Artifacts

Artifact	Type	Tracking Value	Confidence
SHA256 8dfaa1f579...	Hash	Legacy variant (2019-2022) identification	HIGH
Export pattern K7UI_[digit]	Code signature	Masquerade fingerprint; YARA candidate	HIGH
Substitution table byte pattern	Code signature	Three-table sequence - YARA candidate	HIGH
Exception code 0xc0000409	Behavioral	Anti-sandbox technique fingerprint	MODERATE
Indirect call density ~113	Metric	Obfuscation tooling fingerprint	MODERATE
MpSvc.dll string	Artifact	Asia/MENA variant hypothesis	MODERATE
ProxyLogon +1 day timeline	Behavioral	Intelligence access indicator	MODERATE

9. Analysis Limitations

This report is produced with full transparency regarding analytical boundaries. The limitations below are structural constraints that any reader applying this report to operational or policy decisions must understand.

Limitation	Detail
Dynamic analysis incomplete	Stage 2 (MpSvc.dll) was not analyzed. All Stage 2 capability data is sourced from UK NCSC and ESET external reporting.
F17 address pending confirmation	The exact address for the LoadLibraryA DATA reference was not fully captured before the analysis session ended.
Inverted anti-sandbox, operational function unresolved	Three hypotheses remain open (F20). Hypothesis A is most probable but cannot be confirmed without dynamic analysis against live execution.
C2 pivot incomplete	Infrastructure pivot against 194.165.16[.]69 and 198.12.75[.]112 deferred, requires Shodan, Censys, or PassiveTotal access not available at analyst access level.

The conclusions in this report should be understood as the floor of what occurred, not the ceiling.

10. Defensive Recommendations

#	Recommendation
1	DLL sideloading monitoring: alert on K7 Antivirus component names (K7UI.dll, K7UI_1, K7UI_13) loaded from non-standard paths or outside K7 installation directories.
2	IAT scanning limitation: supplement import-table-based detection with behavioral monitoring. Static IAT scanning surfaces only 60 KERNEL32 functions, missing 113 indirect call sites. Behavioral EDR that observes runtime API calls will detect the full footprint.
3	Sandbox evasion: samples terminating immediately with STATUS_STACK_BUFFER_OVERRUN (0xc0000409) on startup should not be classified as non-functional. Re-analyze with a debugger attached - SparrowDoor's inverted logic runs normally under debugger attachment.
4	YARA detection: the three-table substitution byte sequence in .rdata (beginning 0x10008898) is a candidate for a file-based YARA rule with high specificity across FamousSparrow variants.
5	Network monitoring: alert on HTTPS connections to non-approved external destinations from hotel management systems, POS systems, Oracle Opera installations, and legacy Windows infrastructure.
6	Secrets hygiene: do not store API keys, SSH private keys, or plaintext credentials in filesystem locations accessible from development tooling or hotel Wi-Fi networks.
7	ProxyLogon behavioral pattern: monitor for FamousSparrow activity within 24-48 hours of future high-priority CVE patch releases, particularly for Microsoft Exchange or SharePoint.
8	US telecom context: organizations in US telecommunications, ISP, and critical communications infrastructure should treat Salt Typhoon / FamousSparrow intrusion indicators as active, not historical. The 2023-2024 pivot to domestic telecom is confirmed by FBI and CISA joint advisory and assessed as ongoing.

11. References

- [1] ESET Research. FamousSparrow: A suspicious hotel guest. September 2021.
- [2] ESET Research. Updated FamousSparrow analysis - two new SparrowDoor variants. March 2025.
- [3] UK NCSC. Malware Analysis Report: SparrowDoor. February 2022.
- [4] Trend Micro. Earth Estries campaign analysis. November 2024.
- [5] Trustwave SpiderLabs. FamousSparrow / Salt Typhoon report. December 2024.
- [6] Microsoft. Salt Typhoon threat actor profile. 2024.
- [7] FBI / CISA. Joint advisory on Salt Typhoon / FamousSparrow telecom breaches. 2024.
- [8] Wall Street Journal. Salt Typhoon breached multiple US telecommunications providers. September 2024.
- [9] Help Net Security. FamousSparrow 2025 activity update. March 2025.
- [10] MITRE ATT&CK. FamousSparrow (G1101). attack.mitre.org.
- [11] ESET malware-ioc. FamousSparrow sample repository. github.com/eset/malware-ioc/famoussparrow.

TLP:CLEAR. This report may be shared freely without restriction. All network indicators have been defanged.
SerapHim, Independent Cyber Threat Intelligence, March 2026.